Zenith: programs, experiments and useful memory

Will Cook • 15 October 2026

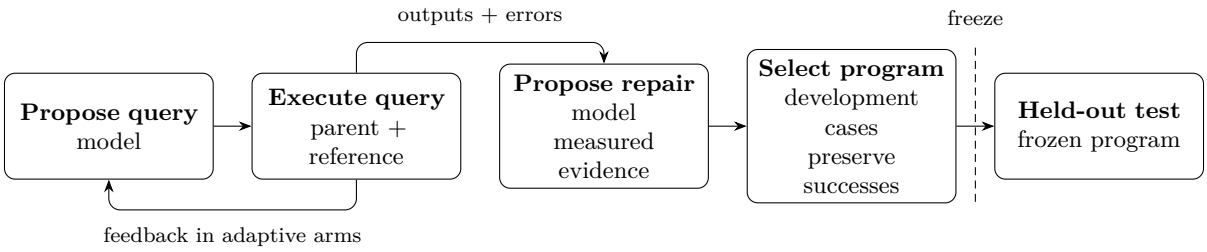
Abstract. We investigate whether retaining programs and their supporting experiments improves performance on subsequent tasks. We evaluate three mechanisms separately: model-driven program repair, reuse of generated testers, and an authored continuation learner. Repair alone increases correct later answers from 259/576 to 365/576; additional investigation does not improve on this baseline. Each of two generated testers detects three of four authored faults, whereas fixed tests detect all four. Retained continuation tests reduce acquisition queries in a controlled ablation, but the learner's total cost exceeds that of ordinary alternatives. These results identify implementation requirements and limitations; they do not establish a new learning algorithm or recursive self-improvement. We outline experiments on transferable investigation procedures and discuss mathematics as a possible evaluation domain.

1. Introduction and experimental setting. We represent retained artifacts as (P, E, Q): a program, source-bound observations, and executable tests or distinguishing continuations. A useful addition must improve later behavior or reduce the work required to acquire it. Cached answers, learned representations and better investigation procedures require separate controls. Model weights remain fixed in these studies.

Predictive state representations encode stochastic state through probabilities of future tests. The cited theorem constructs a representation from a known finite POMDP; it does not supply a black-box learning algorithm [1, Theorem 1; §2]. Our continuation learner instead fits exact integer observations in a rational-linear model. The conceptual connection is the use of future tests to distinguish histories.

Failure diagnosis. A wrong answer may reflect discarded history, an incorrect use of sufficient state, or behavior never observed. The archived SQLite program retained pending inserts but ignored them when reading allocation state; another rows-only helper discarded necessary history. These need different interventions. The records expose the distinction; they do not establish reliable autonomous diagnosis. A model's explanation of a failure remains a hypothesis.

Program repair. In the native study, a model receives a task contract, a whole candidate program, examples and the measurements available to its condition. It proposes queries; the host compares candidate and reference answers. A repair request proposes replacement code. Selection preserves every previously correct measured case and the aggregate development result before later panels open. Rejected proposals and failures remain recorded. Passing these finite checks does not prove general correctness. Lab, Oracle and Evolve name execution, review and revision roles; they need not be separate agents.



Native repair: measured evidence informs the proposed repair. Development selection preserves measured successes and requires a non-decreasing aggregate score. Held-out outcomes are withheld from repair. Tester reuse and continuation learning are separate studies.

Separately, the client study generates and freezes testers before evaluation on later clients with a predeclared interface. A tester proposes queries from supplied context under a fixed driver. The continuation learner is not called by either model-driven study.

2. Experiments and limitations. Continuation learning and counterexample refinement. Access histories h and continuation tests t index an observation table, $H_{h,t} = f(ht)$, for deterministic observations f. Given exact field-valued answers and a square full-rank table, a counterexample supplies a history–test pair whose nonzero residual increases the rank [2, Proposition 1, pp. 2–3]. Our repair preserves the paired access history and distinguishing test required by this construction. It does not discover the model class. Finite challenges replace the global equivalence oracle, so a completed fit establishes agreement only on those challenges. The smallest queue example fits in ten source executions and agrees on lengths 0–32. Its five rational-linear coordinates are relative to the imposed class: one nonlinear integer counter represents that queue more simply.

Across eight inspected native configurations, both the repaired refinement and a generic bank of one-symbol questions complete every fit. Generic questions cost 11,379 acquisition source executions versus 11,395. Later finite checks retain errors: UTF-8 scores 4,903/4,913; SQLite rowid 190/192 and AUTOINCREMENT 191/192. These are diagnostic fixtures, not eight independent applications.

A separate six-teacher study holds cached answers identical. Retaining acquired questions reduces extension queries by 11.2–63.1% in six declared conditions. Yet every joint lifetime costs 5,427–5,698 source executions, versus 3,071 for separate ordinary modules. Blindly transferring all learned suffixes into the simpler learner costs 3,541 and hurts five of six targets. Exact certificates there concern supplied rational-linear references, not arbitrary native software. Useful questions have not yet made the full architecture economical.

Reusable SQLite testers. Five authored batch clients supply development experience. Two generated testers then run unchanged on five account-transfer clients, four faulty and one correct. Each has six query slots per client; invalid and duplicate queries consume slots. Target execution finishes before a separate 400-case panel opens.

Method	Faults / 4	Native probes	Target model calls
Retained tester	3	5	0
Ordinary code reuse	3	5	0
Same-history fresh	3	15	19
Cold investigation	3	15	15
Fixed tester	4	7	0

Both generated testers repeat one constant query, ignore feedback and spend ten slots on duplicates. DeepSeek V4.1 Flash on Morph uses 46 calls for $0.007117764, including development and compilation. All three sequential versions, including two earlier unsuccessful attempts, cost $0.019295552 across 133 calls. The family was inspected during engineering. This establishes executable reuse, not adaptive testing or independent transfer. Charges exclude engineering and host costs; these populations are not pooled with the other studies.

Separate investigation diagnostic. On twelve parent episodes from SQLite, LRU and UTF-8 contracts, repair without new investigation improves confirmation from 259/576 to 365/576 answers. Fixed testing scores 364, compiled investigation 354, a direct batch 339, feedback-aware two-turn requests 343 and outcome-masked requests 331. The paired feedback gain is twelve answers on one parent; eleven parents tie. Only five pairs reach a second decision. Eleven parent programs are unique, but there are only three authored mechanisms; case rows are not independent replications. No investigation arm beats repair alone. Ling3.1Flash on DeepInfra uses 130 calls for $0.12982944. Separate qualification, failed host admission and a funding-only continuation retain their own receipts; no failed attempt is erased or counted twice.

3. Related work and research directions. We propose to retain distinguishing tests, their supporting histories and observations, revised programs, and procedures for selecting informative tests on subsequent tasks. Selective transfer between representation learning and test generation remains unimplemented. The current contribution is an implementation repair and reproducible comparisons with negative controls.

Related work. ILF trains a testing policy from symbolic-execution traces [3, §§4–5]; Zenith does not train model weights. CoverUp guides a fixed LLM with source, coverage and execution feedback to generate regression tests [4, §3]; coverage is not fault detection. Voyager retains executable skills within a fixed acquisition loop [5, §2]. STOP recursively rewrites its code-improvement scaffold [6, §4]. Hyperagents edits task and meta-agent code, then evaluates a fixed acquired meta-agent in another domain. Its reported cross-domain compounding comparison does not establish a statistically significant advantage [7, §§5.2–5.3]. These precedents motivate comparisons with ordinary program and test reuse; Zenith has not established an advantage over these alternatives.

Evaluating an acquired improvement procedure. Let an initial improver produce a successor. Freeze each improver while it edits identical untouched starting solvers on fresh families. Select children on development evidence, freeze their identities, then evaluate once on held-out inputs; never select the best test score. Keep the evaluator outside the edits and count acquisition, failures, evaluation, execution and human intervention. Ablate the learned questions while preserving observations and saved code. Only then compare reinvesting the successor against continuing the original improver. Any gain remains bounded by the tested tasks and budgets.

Mathematical evaluation tasks. The Plectis mathematical corpus contains Lean proofs and open obligations. Its argument-continuation exporter identifies candidate prerequisite theorems and remaining obligations. Plectis Atlas presents OpenAI's mathematics release and attributed outside assessments. Neither is a demonstrated learning integration with Zenith. We would need action, outcome, cost and target relevance records; finished proofs omit alternative attempts. Lean checks supplied proofs; failed search does not establish falsehood. Mathematics is a possible harder transfer domain after cheap software tests.

Collaboration. Zenith is an exploratory project developed with AI coding agents. This account reflects my current understanding as I learn the field. I welcome corrections, collaboration and guidance on prior work, evaluations and useful tasks through the project's issues. Results and replay links retain failures and exact costs; the programme expands the proposed comparisons.

References

[1] M. L. Littman, R. S. Sutton and S. Singh. Predictive Representations of State. NeurIPS 14, 2001.
[2] J. Worrell. Exactly Learning Weighted Automata over a Field. Oxford teaching notes, undated.
[3] J. He et al. Learning to Fuzz from Symbolic Execution with Application to Smart Contracts. CCS, 2019.
[4] J. Altmayer Pizzorno and E. D. Berger. CoverUp: Effective High Coverage Test Generation for Python. PACMSE 2 (FSE), 2025.
[5] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv, 2023.
[6] E. Zelikman et al. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. COLM, 2024.
[7] J. Zhang et al. Hyperagents. arXiv, 2026.